rogii-wellbore-geology-prediction

7位 Solution: HMM + UNet（agent is all you need）

Rank	7
Team	roglike
Author	Gaopeng Ren
Topic ID	733154
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/rogii-wellbore-geology-prediction/discussion/733154
Retrieved with Kaggle CLI on 2026-08-06.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

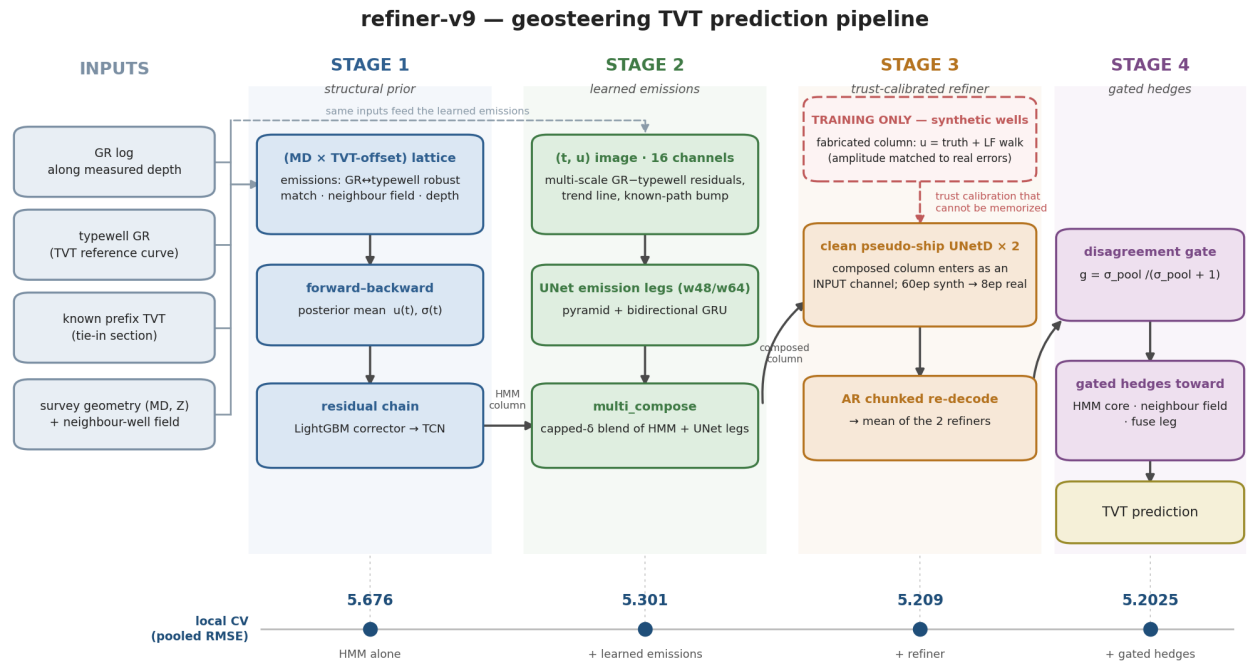
私とteammateの@ejixxxから、主催者と公開Discussion・codeへ貢献した皆様に感謝します。厳しくも興味深い道のりでした。地質知識が全くない中、agentsの助けでこの順位に到達できたことを嬉しく思います。最終的には解法の詳細や動作原理すら十分理解していませんでした。主に行ったのは、agentsのcheatingを止め、新しい発想を得られるresourceを示すことでした。最近の解法の多くが金メダル相当に達したことは、agentsが堅実な解法を提供できることを示しています。学んだのはagentの強力さだけでなく、典型的な間違いと緩和方法です。

1. **Data leakageによるcheat**。たとえばpublic codeのbundleを借り、splitが違うと気づかずlocal solutionへblendします。その後はlocal改善か何度も確認させました。
2. **すぐ諦める**。特にClaude agentsは会話中にcontextをcopyするため、達成不能だと思うほど確信を強めます。対策はnew sessionを始めることだけです。詳細に自信がなく「UNet、Transformerなどを試す」と一般案だけを出すなら、何roundか検証させます。7-8 Public LB域ではUNetが効かなかったのに、5-6域では大きく効きました。
3. **Agentは新しいideaを考案できない**。再現は得意でもinnovationは苦手です。意味のないhyperparameter微調整を無限に続けることを明示的に禁止する必要があります。
4. **Agentには人がresourceを与える必要がある**。Claude agentの`websearch`やliterature searchは賢くなく、似た長い具体keywordを使い続けて結果を狭めがちです。resource、keyword、paperを直接指定しました。

満足できなかったのは、Kaggle向けの統一agent workflowを作れなかったことです。1つのharness（AutoResearch/Tree-based AI Scientistsなど）だけで非常に高い順位へ届くわけではありませんが、人の関与なしでも高性能へ到達できるmethod/workflowはあるはずです。以上がagentを使って解法を構築する方法で、以下はagentsが作った解法の説明です。

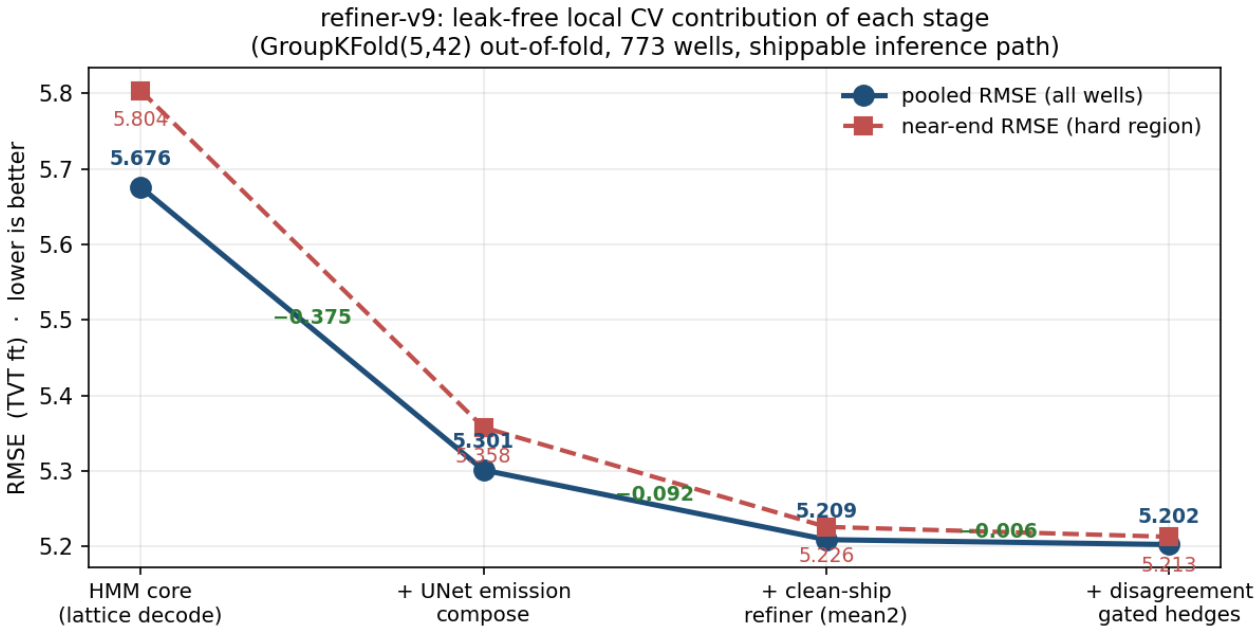
---

1. Pipeline



All scores: GroupKFold(5, seed 42) out-of-fold over 773 wells, computed through the shippable inference path (no post-hoc blend that cannot run on a fresh well).
Deployment: numba-JIT forward-backward (output-exact to 4e-14) · 1-flip decode · CPU-warm/GPU-decode overlap → T4 5 m 54 s | public 5.518 · private 6.057 · 7th / 6191

すべてのstageを同じ方法で評価します。**全773 training wellsに対するGroupKFold(5, seed 42) OOFで、fresh unseen well上で動くshippable inference pathを通す**方法です。推論で再構築できないpost-hoc blendは使いません。



HMM単体は5.676。learned emissionsで5.301。最も調整に時間がかかったrefiner stageで5.209、gated hedgesを加えた最終は**5.2025 pooled / 5.2128 near-end**です。

---

2. Stage 3を機能させたidea: 作り物の失敗でpretrainする

refinerは、first-pass predictionを追加input channelとして受け取り、それを直すsecond-pass UNetです。real wellのOOF列を入力してfine-tuneする明白な方法は失敗しました。診断が重要です。

> fold netは**自分のtraining wellsをRMSE 3.20**でdecodeする一方、held-out wellsでは**9.34**でした。

training中のconditioning channelがほぼ完璧なので、lossを下げる最短手段はそれをcopyすることです。training distributionでは入力がほぼ間違わないため、いつ間違うかを学べず、memorizationでtrust calibrationが壊れます。対策は、**memorizeできないwellでtrustを学ぶこと**です。約6,000 synthetic wellsを生成し、conditioning channelを次のように作ります。

```
u_ship = truth + amplitude × low-passed random walk        (tie-inでanchor)
amplitude ~ lognormal(median ≈ 2.9, σ = 0.68)              (測定したreal error distributionに一致)
15% of wells drawn as "good ship" (amplitude 0.3–1.0)
```

これを60 epochs pretrainし、real wellは8 epochsだけfine-tuneします。入力列が現実的な形で自信ありげに間違うwellを何千も見せることで、どこで疑うかを学び、短いreal-data fine-tune後も残りました。pretrained stateはconditioning generationをまたいで再利用でき、後続campaignはrefiner variantあたり10分と安価になりました。v9はplain版2つを平均します。

---

3. Gated-signal principle

役に立たないように見えたauxiliary signalがいくつかありました。neighbour-well field predictionとcomposed column residualの相関は`r ≈ -0.02`で、flat blendでは何も起きません。しかし2つのrefinerは不確かな場所で正確に食い違います。per-node standard deviationをgate `g = σ/(σ+1)`として、hedgeをgate経由だけで適用します。

```
p ← mean(refiners)
p ← p + 0.10·g·(HMM core − p)          structural priorへshrink
p ← p + 0.15·g·clip(neighbour − p)     「死んだ」signalを復活
p ← p + 0.15·g·clip(fuse leg − p)
p ← p + 0.05·g·clip(neighbour − p)
```

各stage単体の価値は小さいものの、gateを通して合計するとpooledで0.006、hard near-end wellsで0.013改善しました。一般則は、平均では無用なsignalも、ensembleが迷っている場所でだけ役立つ場合があるということです。

---

4. 「wall-survival」と呼ぶ理由

直前24時間の提出はすべて**得点されず、9-hour runtime wallに当たりました**。hidden setはnotebookが見る3-well stubよりずっと大きいため、拘束指標はstub timeではなくseconds-per-wellです。v9は生き残るようengineerしたversionです。

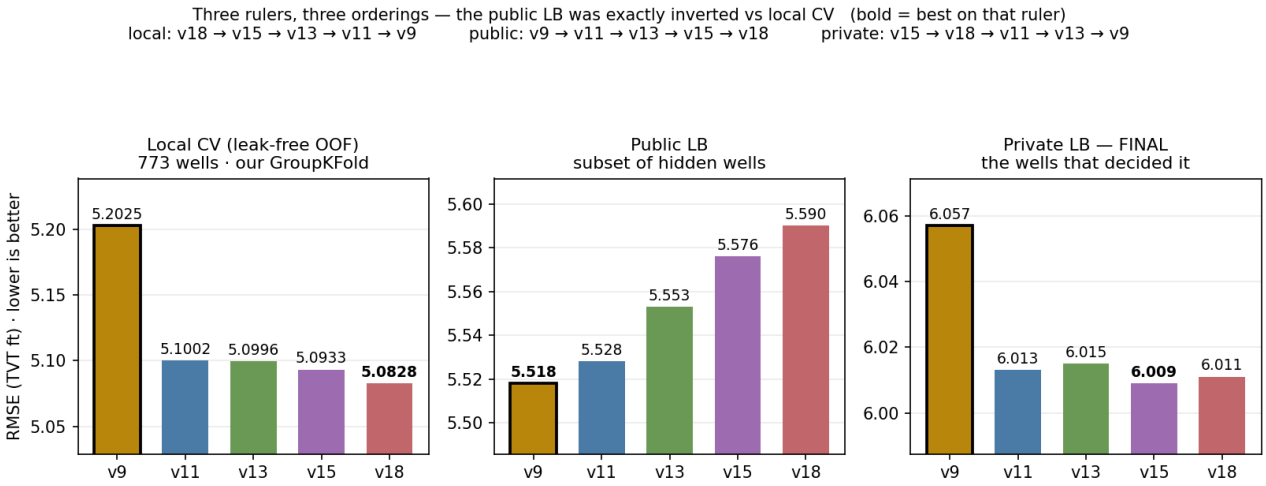
- HMMの**numba-JIT forward-backward**: banded tridiagonal transition kernel、unique-emission-row exponentials、float64 running state。decodeを25 s→12 sへ短縮し、ship前にnumpy pathとの差を確認して**4e-14までoutput-exact**。
- main/fuse legsを**1-flip decode**へし、flip-TTAを落としました。精度costを意図的に受け入れています。
- **CPU-warm / GPU-decode overlap**: forkserver workersのwarm-up中にprefetch threadが最初のwellのUNet legsをdecode。
- fp16 autocast + NaN guards、familyごとのdecode chunking、batched pip installs、model load周辺で`gc.disable()`。

結果は**T4 stub 5m54s、hidden wellあたり約50-65 s**で、予想より数倍大きいhidden setにも余裕がありました。得点され、後発のより複雑なvariantはこのruntime基盤上に構築しました。

---

5. 正直な追記: どの物差しが真実だったか

洗練度が上がる5つのconstructionでコンペを終えました。bestがどれかは、どの物差しに聞くかで変わります。



- **Local CV:** v18 best、v9は明確にworst（5.083 vs 5.203）。
- **Public LB:** 完全に逆でv9 best（5.518）、v18 worst（5.590）。
- **Private LB:** 結局local CVがほぼ正しく、v9は最下位（6.057）、enriched constructionsは6.009-6.015へ集中。

追加pool enrichmentがOOF error structureへfitし転移しないと考え、Public LBを理由にv9を選びました。773 wellsのpaired bootstrapでは、Public inversionがsampling noiseである確率は1.75%しかなく、それを信じました。

これは示唆的な誤りでした。Public splitは順位を完全に逆転させるほど小さく、bootstrapが測ったのは自分たちのwell distributionであってPublic splitではありません。leak-free local CVは最初から順位を正しく示していました。

損失にはなりませんでした。Privateで5案のspreadは0.048で、どれでも順位は同じく7位でした。次のコンペへのtakeawayは明確です。**773 wellsを使う厳密にleak-freeなlocal CVは、その一部だけのPublic scoreより強く、Public LBの小差は証拠ではありません。** Public LBが信頼できたのはconstant offset（local + 約0.31）で、0.1幅内の順位ではありませんでした。